Objective:

The objective of this project is to explore process creation, termination, and **shared memory inter-process communication**. To that end, we calculate the sum of all elements in a matrix by increasing the number of cooperating processes from 1 to 256 in power of 2 and measure the total execution time for each process count.

Project Description:

Consider an M x N matrix A, where M is number of rows and N is number of columns. Each element of the matrix holds an integer value. The value of element indexed at row i and column j is set to i+j i.e.,

$$A_{i,j} = i + j$$

Assuming C number of cooperating processes, each process is responsible for summing M/P rows. To be more specific,

- process p_0 calculates partial sum of all elements of first M/P rows
- process p_1 calculates partial sum of all elements of next M/P rows
-
- process p_{C-1} calculates partial sum of all elements of last M/P rows.

A parent process creates two shared memory segments:

Memory Segment 1 (MS1) is large enough to hold a matrix of size M x N. Parent processes initializes elements of matrix in MS1 according to the equation shown above.

Memory Segment 2 (MS2) is large enough to hold C number of partial sums (of type long) calculated by C number of children processes.

The parent process then launches C number of children processes and assigns each one of them M/P rows to work on and specifies the index of MS2 where a child is supposed to store the partial sum of assigned rows. Note that each child process runs a new binary (i.e., binary of **child_proc.c**). After launching all children processes, the parents wait for them to complete.

Given Files

Following files, containing code skeleton for the project are provided.

1. **Project1_Module_B.h**
2. **Project1_Module_B.c**
3. **child_proc.c**

4. **Makefile**
5. **Graph.xlsx**

Required:

1. Complete the skeleton code by writing all the missing parts (indicated by ToDo).
    a. 7 missing parts in **Project1_Module_B.c**
    b. 1 missing part in **Project1_Module_B.h**
    c. 1 missing part in **child_proc.c**
2. After completing the missing parts, compile the code by running "make". Note that I tested the solution by using the given Makefile on Ubuntu 24.04. It might not work on MacOS.

3. Run the executable file of "parent" with number of processes set to 1,2,4,8,16,32,64,128,256, 512, 1024 (For example ./parent 16)
    a. For each process count, record the execution time printed in seconds

4. Draw a graph by filling the column B, C, and D (by running your program three times for a given number of process count) of the given excel sheet.
5. Find number of cores on your computer (on Ubuntu you can do "lscpu" on terminal to find number of cores). Write 4-5 sentences to explain the graph, any increase/decrease in normalized execution time with increasing number of process count, the relationship of the increase/decrease with number of cores.

Deliverables.

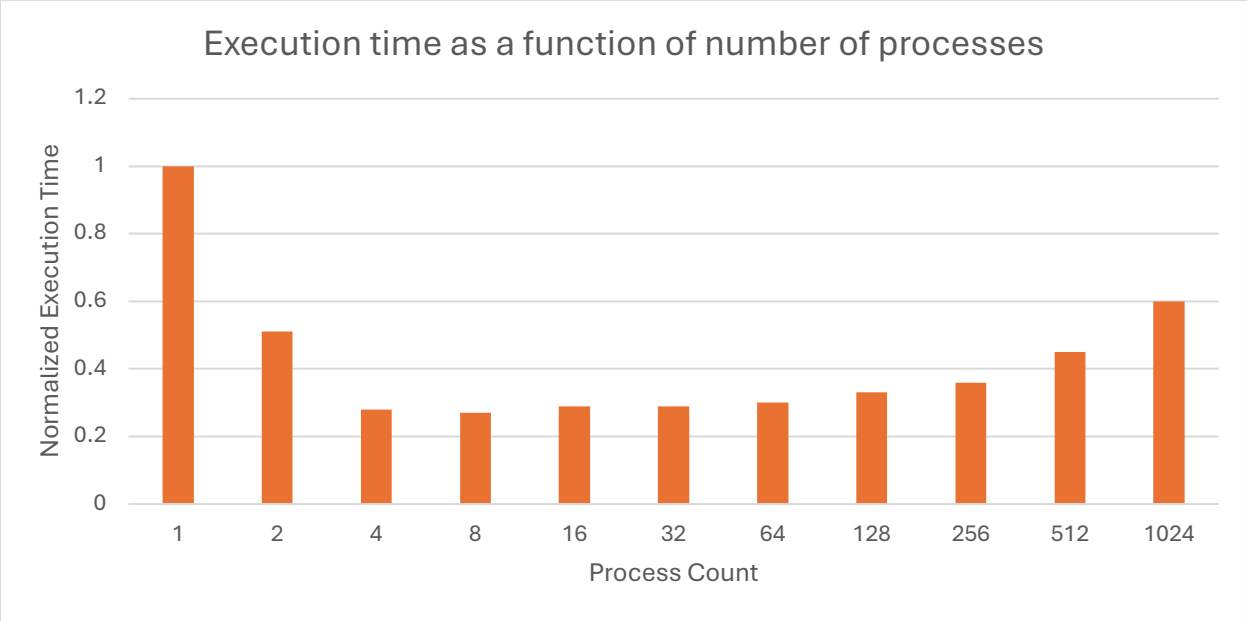
1. **Complete Source Code:** For all three skeleton files
    a. **Evaluation:** 60 points for the working code. No points for incomplete code.
2. **Filled Excel Sheet with Graph**
    a. **Evaluation:** 20 points for drawing the graph
3. **4-5 line explanation of the graph.**
    a. **Evaluation:** 20 points for the explanation.

Submission Guidelines:

Place all .c, .h, excel sheet, and .doc file (for graph explanation) in a single folder, name it as First_Student_ID_First_Student_Name_Second_Student_ID_Second_Student_Name, zip it, and submit.

Sample Output:

Following table and the graph shows the measurements for a matrix of size 16384 x 16384, where number of processes is increased from 1 to 1024.



Process Count	Execution Time (Sec)				Normalized Execution Time
	run1	run2	run3	average	
1	0.264199	0.199445	0.208301	0.22398167	1
2	0.113551	0.109855	0.11712	0.11350867	0.51
4	0.063642	0.064758	0.058131	0.062177	0.28
8	0.060441	0.058625	0.059041	0.059369	0.27
16	0.063405	0.065421	0.062702	0.06384267	0.29
32	0.071135	0.061033	0.061743	0.064637	0.29
64	0.06361	0.069578	0.069563	0.06758367	0.3
128	0.071034	0.080972	0.068588	0.07353133	0.33
256	0.076371	0.09046	0.075025	0.08061867	0.36
512	0.089065	0.114942	0.101113	0.10170667	0.45
1024	0.130761	0.130858	0.143785	0.13513467	0.6